

ResolveAI

Production-Grade AI-First Full-Stack Project Blueprint

React, Next.js, Node.js, Express, FastAPI, LangChain, LangGraph, LangSmith, RAG, Docker, CI/CD,
AWS

Prepared for: AI-First Full-Stack Engineer Portfolio

Purpose: Build a real product from scratch, deploy it, and use it to apply for jobs.

Table of Contents

1. 1. Executive Summary
2. 2. Product Idea and Market Positioning
3. 3. Target Users, Problems, and Solution
4. 4. Complete Tech Stack Responsibilities
5. 5. Main Product Modules
6. 6. System Architecture
7. 7. Monorepo and Folder Structure
8. 8. Database Design
9. 9. API Design
10. 10. RAG Pipeline Design
11. 11. Agentic Workflow Design with LangGraph
12. 12. Human Approval and Safety Layer
13. 13. Frontend Web App Screens
14. 14. React Native Mobile App Scope
15. 15. DevOps, Docker, CI/CD, and AWS Deployment
16. 16. Observability with LangSmith and Logs
17. 17. Security and Production Requirements
18. 18. Testing, Evaluation, and Quality Gates
19. 19. MVP Scope and Advanced Roadmap
20. 20. 30-Day Build Plan
21. 21. Demo Scenarios for Recruiters
22. 22. Portfolio, README, LinkedIn, and Resume Positioning
23. 23. Starter Commands and Environment Variables
24. 24. Final Checklist

1. Executive Summary

ResolveAI is a production-grade agentic support and incident resolution platform. It is designed as a real SaaS product for software companies, support teams, DevOps teams, and agencies. The system allows a company to upload internal documentation, run source-grounded AI chat, triage tickets, generate customer replies, create GitHub issues, and guide teams during incidents using human-approved AI workflows.

This project is intentionally designed to prove full-stack engineering, AI engineering, system design, and DevOps skills together. It is not a simple chatbot. It combines multi-tenant SaaS architecture, RAG pipelines, agentic workflows, role-based access control, observability, Docker deployment, and AWS infrastructure.

Area	Details
Product name	ResolveAI
Product category	Agentic AI SaaS for support and incident resolution
Main users	SaaS companies, software houses, support teams, DevOps teams, agencies
Core value	Reduce support resolution time by connecting company knowledge, tickets, incidents, and developer workflows through AI agents.
Your positioning	AI-first full-stack engineer who can build complete production-grade AI products.

Why this project is strong for jobs

- It uses React and Next.js for a professional dashboard and AI chat interface.
- It uses Node.js and Express for the main SaaS backend, authentication, teams, tickets, integrations, and business APIs.
- It uses Python FastAPI for the AI service, RAG pipeline, LangChain, LangGraph, and LangSmith tracing.
- It shows system design skills: microservices, queues, vector search, file storage, caching, observability, deployment, and tenant isolation.
- It has real business value. Companies actually need faster support, better incident handling, and internal knowledge search.
- It is easy to demo to recruiters because the workflow is visual and practical.

2. Product Idea and Market Positioning

Many developers build AI chatbots, but a chatbot alone is not enough to prove production AI engineering. ResolveAI should be built as a workflow product where the AI can retrieve knowledge, reason over tickets, call tools, request human approval, and produce useful outputs for a team.

Product one-liner

ResolveAI is an AI support and incident resolution platform that helps SaaS teams answer customer questions, triage tickets, debug incidents, and create developer tasks using company knowledge and human-approved AI agents.

Product promise

- Turn scattered company knowledge into a searchable AI support brain.
- Help support agents reply faster with source-grounded answers.
- Help developers understand incidents using past tickets, logs, runbooks, and GitHub issues.
- Keep humans in control before any external action is executed.
- Give admins analytics about AI usage, cost, confidence, and knowledge gaps.

Business model direction

- Free tier: limited knowledge sources, limited messages, one organization.
- Pro tier: team members, more documents, ticket triage, GitHub integration.

- Business tier: advanced workflows, analytics, approval system, Slack integration, priority support.
- Agency mode: agencies can manage multiple client workspaces.

3. Target Users, Problems, and Solution

User type	Problem	ResolveAI solution
Support agent	Wastes time searching docs and past tickets.	AI chat answers with citations and suggests replies.
Support manager	Hard to measure common issues and response quality.	Analytics dashboard shows categories, escalations, and AI performance.
Developer	Receives unclear bug reports from support.	AI creates structured GitHub issues with reproduction steps and evidence.
DevOps/on-call engineer	Needs fast incident context during production issues.	Incident agent searches runbooks, similar incidents, and suggests next steps.
Founder/agency owner	Needs support quality without hiring a large team.	AI helps small teams handle customer support professionally.

Main problem

Software companies keep knowledge in many places: docs, GitHub, tickets, logs, Slack, Notion, PDFs, FAQs, deployment notes, runbooks, and old incident reports. When a customer reports a problem, the team loses time finding the right answer. When a production incident happens, the team needs context quickly.

Main solution

ResolveAI centralizes company knowledge and connects it to AI workflows. The AI does not only generate text. It retrieves relevant sources, reasons over the problem, classifies tickets, suggests next actions, drafts replies, creates structured developer tasks, and records everything for observability and audit.

Core jobs-to-be-done

- When a support question arrives, I want an accurate answer with sources so I can reply quickly.
- When a ticket is unclear, I want AI to classify it and suggest priority so the team can act faster.
- When a bug is reported, I want AI to create a clean developer issue so engineering can reproduce it.
- When an incident happens, I want AI to guide the on-call person using runbooks and past incidents.
- When AI takes action, I want approval controls so the system remains safe.

4. Complete Tech Stack Responsibilities

Use every technology with a clear responsibility. Do not add tools only because they look good on the resume.

Layer	Technology	Responsibility
Frontend web	React + Next.js + TypeScript	Dashboard, auth screens, knowledge upload, chat UI, ticket system, analytics.
Mobile app	React Native	On-call alerts, approval actions, critical tickets, quick summaries.
Main backend	Node.js + Express + TypeScript	Auth, SaaS APIs, organizations, tickets, integrations, audit logs, API gateway.
AI backend	Python + FastAPI	RAG ingestion, embeddings, LangChain chains, LangGraph agents, streaming responses.
AI orchestration	LangGraph	Controlled agent workflows, nodes, conditional edges, approval checkpoints.
LLM application framework	LangChain	Retrievers, prompts, output parsers, tool wrappers, chains.

LLM observability	LangSmith	Tracing, debugging, cost, latency, evaluation, agent run inspection.
Database	PostgreSQL	Users, organizations, tickets, messages, sources, audit logs, agent runs.
Vector database	pgvector or Qdrant	Store embeddings and perform similarity search.
Queue/cache	Redis + BullMQ	Background jobs, ingestion pipeline, retries, rate limits.
File storage	AWS S3	Uploaded PDFs, markdown files, raw source documents.
Deployment	Linux + Docker + Nginx + AWS	Run services in containers and expose them with HTTPS.
CI/CD	GitHub Actions	Test, build Docker images, deploy automatically.

Why use both Node.js and FastAPI?

- Node.js should be the product backend because it is excellent for web APIs, auth, user management, real-time events, and integrations.
- FastAPI should be the AI backend because Python has the strongest AI ecosystem for LangChain, LangGraph, embeddings, document loaders, and evaluation.
- The Node API calls the FastAPI AI service internally. The frontend does not directly talk to the AI service except through secure API gateway patterns.

5. Main Product Modules

Module 1: Authentication and multi-tenant SaaS

- Register and login users.
- Create organizations/workspaces.
- Invite team members.
- Roles: Owner, Admin, Support Agent, Developer, Viewer.
- JWT access tokens and refresh tokens.
- Tenant isolation using organization_id in every business table.
- Protected frontend routes and API middleware.

Module 2: Knowledge base

- Upload PDF, Markdown, TXT, and DOCX files.
- Add website URLs or GitHub README links.
- Extract text and metadata.
- Chunk text and generate embeddings.
- Store source records and ingestion status.
- Allow re-indexing and deleting sources.
- Show source health and last indexed date.

Module 3: AI support chat

- Chat with company knowledge.
- Stream answer token by token.
- Show source citations.
- Show confidence score.
- Show related documents.
- Save conversation history.
- Allow creating a ticket from a chat.
- Fallback when answer is not found in sources.

Module 4: Ticket triage agent

- Classify ticket category.
- Detect priority and urgency.
- Detect sentiment.
- Find similar past tickets.
- Suggest assigned team role.
- Draft customer reply.
- Suggest escalation when needed.

Module 5: Incident resolution agent

- Understand incident description.
- Search runbooks and past incidents.
- Ask for missing logs or context.
- Suggest likely causes.
- Suggest rollback or fix checklist.
- Generate incident summary.
- Create postmortem draft.
- Require approval for external actions.

Module 6: GitHub integration

- Connect GitHub repository.
- Read repository metadata and README.
- Create GitHub issue from ticket.
- Generate issue title, description, reproduction steps, and investigation checklist.
- Link ticket to created issue.
- Show GitHub issue status in ResolveAI.

Module 7: Agent workflow builder

- Admin creates workflow rules.
- Example: when new ticket arrives, classify, retrieve, draft reply, then send for approval.
- Support workflow run history.
- Show current node and tool calls.
- Allow retrying failed runs.

Module 8: Analytics dashboard

- Total tickets.
- Resolved by AI.
- Escalated to human.
- Average response time.
- AI confidence score trends.
- Token usage and LLM cost.
- Failed agent runs.
- Top knowledge gaps.

6. System Architecture

Use a service-based architecture. Keep product logic and AI logic separated but connected through internal APIs.

```
Users
|
|-- Next.js Web App
|-- React Native Mobile App
|
v
Node.js Express API Gateway
|-- Auth and RBAC
|-- Organization and team management
|-- Tickets and conversations
```

```

|-- Integrations and audit logs
|-- Calls FastAPI AI service
|
|-- PostgreSQL: product data
|-- Redis: queues, cache, rate limits
|-- S3: uploaded source files
|
v
FastAPI AI Service
|-- RAG ingestion pipeline
|-- Embeddings and vector search
|-- LangChain chains
|-- LangGraph agent workflows
|-- Tool calling
|-- Streaming responses
|-- LangSmith tracing
|
|-- Vector DB: pgvector or Qdrant
|-- LLM provider
|-- Embedding provider
|
v
External tools
|-- GitHub
|-- Slack or email later
|-- AWS services

Deployment
|-- Docker containers
|-- Nginx reverse proxy
|-- GitHub Actions CI/CD
|-- AWS EC2 or ECS
|-- RDS PostgreSQL
|-- CloudWatch logs

```

Recommended first deployment architecture

- Use one AWS EC2 instance first to reduce complexity.
- Run Next.js, Node API, FastAPI, PostgreSQL, Redis, and Qdrant/pgvector with Docker Compose.
- Use Nginx as reverse proxy.
- Use Certbot for HTTPS.
- After MVP, move PostgreSQL to RDS and file storage to S3.

7. Monorepo and Folder Structure

```

resolve-ai/
|
|-- apps/
|   |-- web/                # Next.js frontend
|   |-- mobile/             # React Native app later
|
|-- services/
|   |-- api/                # Node.js Express backend
|   |   |-- src/
|   |   |   |-- modules/
|   |   |   |   |-- auth/
|   |   |   |   |-- organizations/
|   |   |   |   |-- tickets/
|   |   |   |   |-- knowledge/
|   |   |   |   |-- conversations/
|   |   |   |   |-- agent-runs/
|   |   |   |   |-- approvals/
|   |   |   |   |-- integrations/
|   |   |   |-- middleware/
|   |   |   |-- config/
|   |   |   |-- utils/
|   |   |-- prisma/
|   |-- Dockerfile

```

```

| | -- ai-service/                                # FastAPI + LangChain + LangGraph
| | | -- app/
| | | | -- main.py
| | | | -- api/
| | | | -- rag/
| | | | -- agents/
| | | | -- tools/
| | | | -- prompts/
| | | | -- evaluators/
| | | | -- settings.py
| | | -- Dockerfile
|
| -- packages/
| | -- shared-types/                            # Shared TypeScript types
| | -- ui/                                       # Shared UI package if needed
|
| -- infra/
| | -- nginx/
| | -- docker/
| | -- github-actions/
| | -- aws/
|
| -- docs/
| | -- system-design.md
| | -- api-docs.md
| | -- rag-pipeline.md
| | -- agent-workflows.md
| | -- deployment.md
|
| -- docker-compose.yml
| -- README.md
| -- .env.example
| -- .github/workflows/

```

8. Database Design

Use PostgreSQL for product data. For vectors, start with pgvector if you want one database, or use Qdrant if you want a separate vector service.

Core tables

```

users
- id
- name
- email
- password_hash
- avatar_url
- created_at
- updated_at

organizations
- id
- name
- slug
- plan
- created_at
- updated_at

organization_members
- id
- user_id
- organization_id
- role
- status
- invited_by
- created_at

knowledge_sources
- id

```



```

- organization_id
- type                # pdf, markdown, txt, website, github
- name
- status              # pending, processing, indexed, failed
- url
- file_path
- metadata_json
- last_indexed_at
- created_by
- created_at

documents
- id
- source_id
- organization_id
- title
- content_hash
- raw_text_preview
- metadata_json
- created_at

document_chunks
- id
- document_id
- organization_id
- chunk_index
- chunk_text
- token_count
- embedding_id        # or vector stored directly if using pgvector
- metadata_json
- created_at

tickets
- id
- organization_id
- customer_name
- customer_email
- title
- description
- status              # open, in_progress, resolved, closed
- priority            # low, medium, high, critical
- category
- sentiment
- assigned_to
- source
- created_at
- updated_at

conversations
- id
- organization_id
- ticket_id
- user_id
- title
- created_at

messages
- id
- conversation_id
- role                # user, assistant, system, tool
- content
- sources_json
- metadata_json
- created_at

agent_runs
- id
- organization_id
- conversation_id
- ticket_id
- workflow_name

```

```

- status                # running, completed, failed, waiting_approval
- input_json
- output_json
- cost_usd
- latency_ms
- langsmith_trace_url
- created_at

tool_calls
- id
- agent_run_id
- tool_name
- input_json
- output_json
- status
- error_message
- created_at

approvals
- id
- organization_id
- agent_run_id
- action_type
- proposed_action_json
- status                # pending, approved, rejected
- approved_by
- decided_at
- created_at

audit_logs
- id
- organization_id
- user_id
- action
- entity_type
- entity_id
- metadata_json
- ip_address
- created_at

```

Important database rules

- Every organization-specific table must include `organization_id`.
- Every API query must filter by `organization_id` from the authenticated user context.
- Do not trust frontend `organization_id` blindly. Resolve it from membership and auth context.
- Use indexes on `organization_id`, ticket status, `created_at`, and document `source_id`.
- Keep audit logs for important actions like upload, delete source, approve action, create GitHub issue, and invite member.

9. API Design

Node.js Express APIs

```

Auth
POST  /api/auth/register
POST  /api/auth/login
POST  /api/auth/refresh
POST  /api/auth/logout
GET   /api/me

Organizations
POST  /api/orgs
GET   /api/orgs/current
PATCH /api/orgs/current
POST  /api/orgs/invite
GET   /api/orgs/members
PATCH /api/orgs/members/:memberId/role

```

Tickets

```
GET    /api/tickets
POST   /api/tickets
GET    /api/tickets/:id
PATCH /api/tickets/:id
POST   /api/tickets/:id/triage
POST   /api/tickets/:id/suggest-reply
```

Knowledge

```
POST   /api/knowledge/upload
POST   /api/knowledge/url
GET    /api/knowledge/sources
GET    /api/knowledge/sources/:id
POST   /api/knowledge/sources/:id/reindex
DELETE /api/knowledge/sources/:id
```

Chat and conversations

```
POST   /api/chat
POST   /api/chat/stream
GET    /api/conversations
GET    /api/conversations/:id/messages
```

Agent runs and approvals

```
POST   /api/agent/run
GET    /api/agent/runs
GET    /api/agent/runs/:id
GET    /api/approvals
POST   /api/approvals/:id/approve
POST   /api/approvals/:id/reject
```

Integrations

```
POST   /api/integrations/github/connect
GET    /api/integrations/github/repos
POST   /api/integrations/github/issues
POST   /api/integrations/slack/connect # later
```

FastAPI AI service APIs

Health

```
GET    /ai/health
```

Ingestion

```
POST   /ai/ingest/file
POST   /ai/ingest/url
POST   /ai/ingest/github-readme
```

Retrieval

```
POST   /ai/retrieve
POST   /ai/rerank
```

Chat

```
POST   /ai/chat
POST   /ai/chat/stream
```

Agents

```
POST   /ai/agent/support-triage
POST   /ai/agent/incident-resolution
POST   /ai/agent/draft-reply
POST   /ai/agent/create-github-issue-draft
```

Evaluation

```
POST   /ai/evaluate/rag-answer
POST   /ai/evaluate/test-dataset
```

Internal callbacks

```
POST   /ai/internal/approval-result
```

API gateway rule

The frontend should call the Node.js API. The Node.js API then calls the FastAPI AI service. This keeps authentication, tenant checks, rate limits, and audit logs centralized.

10. RAG Pipeline Design

The RAG pipeline is the foundation of ResolveAI. It must be reliable, source-grounded, and auditable.

Ingestion flow

25. User uploads a file or adds a URL in the Next.js dashboard.
26. Node API validates file type, organization membership, and file size.
27. Node API stores raw file in S3 or local storage for MVP.
28. Node API creates a knowledge_sources record with status = pending.
29. Node API sends ingestion job to FastAPI or queues it through Redis/BullMQ.
30. FastAPI extracts text from the document.
31. FastAPI cleans text and removes repeated headers/footers where possible.
32. FastAPI chunks text using semantic or recursive chunking.
33. FastAPI creates embeddings for each chunk.
34. FastAPI stores chunks and vector references.
35. Node API updates source status to indexed or failed.

Retrieval flow

36. User asks a question.
37. AI service rewrites the query for better retrieval.
38. Retriever searches vector DB with organization_id filter.
39. Optional keyword search is combined with vector search.
40. Optional reranker ranks top chunks.
41. Evidence grader checks if retrieved context is enough.
42. LLM generates answer only from selected evidence.
43. System returns answer, sources, confidence, and follow-up suggestions.

RAG answer policy

- Use only retrieved company sources for factual answers.
- If sources are weak, say that the knowledge base does not contain enough information.
- Always return source references.
- Always include confidence score.
- Never invent company policy, prices, code behavior, or incident history.
- For risky actions, suggest next step and request approval.

Chunking recommendation

- Start with 800 to 1200 token chunks.
- Use 100 to 200 token overlap.
- Store metadata: document title, page number, section heading, source id, file type, and organization id.
- For FAQs, keep question-answer pairs together.
- For runbooks, keep step-by-step procedures together.

11. Agentic Workflow Design with LangGraph

LangGraph should be used when the task needs decisions, tool calls, retries, conditional routing, or human approval. Simple RAG questions can use a simpler chain, but incident resolution and ticket triage should use LangGraph.

Main support workflow

```
User question or ticket
|
v
Intent classifier
|-- general_question
```

```

|-- support_ticket
|-- bug_report
|-- incident
|-- action_request
v
Query rewriter
v
Retriever
v
Evidence grader
|-- enough_context -> Answer generator
|-- not_enough_context -> Clarifying question generator
|-- action_needed -> Tool planner
v
Tool planner
|-- draft_reply
|-- create_ticket
|-- create_GitHub_issue_draft
|-- escalate_to_human
v
Human approval node if needed
|-- approved -> execute tool
|-- rejected -> explain and stop
v
Final response + audit log

```

Agent state design

```

AgentState
- organization_id
- user_id
- ticket_id
- conversation_id
- input_text
- intent
- rewritten_query
- retrieved_docs
- evidence_score
- confidence
- planned_action
- approval_required
- approval_id
- tool_results
- final_answer
- errors
- trace_id

```

Recommended LangGraph nodes

Node	Purpose	Output
intent_classifier	Understand user intent.	intent label and reasoning summary
query_rewriter	Improve the search query.	rewritten query
retriever	Fetch relevant chunks.	retrieved documents
evidence_grader	Check if context is enough.	score and decision
answer_generator	Generate source-grounded answer.	answer with citations
tool_planner	Decide if an action is needed.	planned tool call
approval_gate	Pause risky action for human approval.	approval request
tool_executor	Execute approved tool.	tool result
finalizer	Return final response and save trace.	final assistant message

Agent tools

```

search_knowledge_base(query, organization_id)

```

```

search_similar_tickets(query, organization_id)
read_ticket(ticket_id, organization_id)
create_ticket(payload, organization_id)
update_ticket(ticket_id, payload, organization_id)
draft_customer_reply(ticket_id, context)
create_github_issue_draft(ticket_id, context)
create_github_issue_after_approval(payload)
escalate_to_human(ticket_id, reason)
summarize_incident(incident_context)
generate_postmortem(incident_context)

```

12. Human Approval and Safety Layer

A production agent should not blindly perform external actions. ResolveAI should make AI powerful but controlled.

Actions that require approval

- Sending a customer reply.
- Creating a GitHub issue.
- Updating ticket status to resolved.
- Posting to Slack or email.
- Running any command or deployment-related action.
- Deleting or re-indexing a knowledge source.

Approval lifecycle

44. Agent proposes an action.
45. System stores proposed action in approvals table.
46. Frontend shows approval card to allowed roles.
47. Human clicks approve or reject.
48. Node API writes audit log.
49. If approved, Node API or FastAPI executes the tool.
50. Result is attached to the ticket and agent run.

```

Approval card example
Action: Create GitHub issue
Ticket: Payment success but subscription inactive
AI confidence: 0.86
Repository: company/backend
Proposed title: Payment webhook not activating subscription
Buttons: Approve | Reject | Edit before approval

```

13. Frontend Web App Screens

The frontend should feel like a SaaS product. Use clean dashboard layouts, cards, filters, tables, and real-time AI interactions.

Screen	Main components	Purpose
Landing page	Hero, features, demo screenshots, CTA	Show product value publicly.
Login/register	Forms, validation, error states	User onboarding.
Organization setup	Create workspace, invite team	Multi-tenant SaaS flow.
Dashboard	Stats cards, recent tickets, AI usage	Quick overview.
Knowledge base	Upload source, source list, status badges	Manage indexed docs.
AI chat	Chat panel, source cards, confidence score	Ask questions from company knowledge.
Tickets	Table, filters, status, priority	Support ticket management.
Ticket detail	Conversation, AI triage, suggested reply	Resolve one ticket.
Agent runs	Timeline, nodes, tool calls, errors	Debug AI workflow.

Approvals	Pending action cards	Human-in-the-loop control.
Integrations	GitHub connect, repo selector	External workflow connection.
Analytics	Charts, cost, latency, common issues	Operational insight.
Settings	Members, roles, API keys, plan	Admin management.

AI chat UI requirements

- Streaming answer.
- Source cards below every answer.
- Confidence score indicator.
- Button: Create ticket.
- Button: Draft reply.
- Button: Escalate.
- Conversation history sidebar.
- Empty state with example questions.

14. React Native Mobile App Scope

The mobile app should be a companion app, not a full replacement for the web dashboard. Build it after the web MVP is stable.

Mobile app use cases

- On-call support alerts.
- View critical tickets.
- Read AI-generated incident summary.
- Approve or reject AI actions.
- Receive push notification for high-priority issues.
- Quick chat with company knowledge.
- Voice note to incident summary later.

Mobile screens

- Login screen.
- Critical tickets list.
- Ticket detail screen.
- AI summary screen.
- Approvals screen.
- Push notification settings.

15. DevOps, Docker, CI/CD, and AWS Deployment

Local Docker Compose services

```
services:
  web:
    build: ./apps/web
    ports:
      - "3000:3000"
    depends_on:
      - api
  api:
    build: ./services/api
    ports:
      - "4000:4000"
    env_file:
```

```

- .env
depends_on:
- postgres
- redis
- ai-service

ai-service:
  build: ./services/ai-service
  ports:
  - "8000:8000"
  env_file:
  - .env
  depends_on:
  - postgres
  - redis
  - qdrant

postgres:
  image: pgvector/pgvector:pg16
  ports:
  - "5432:5432"
  environment:
    POSTGRES_DB: resolveai
    POSTGRES_USER: resolveai
    POSTGRES_PASSWORD: resolveai

redis:
  image: redis:7
  ports:
  - "6379:6379"

qdrant:
  image: qdrant/qdrant
  ports:
  - "6333:6333"

```

AWS MVP deployment

- EC2 Ubuntu server.
- Install Docker and Docker Compose.
- Nginx reverse proxy.
- Domain connected to EC2 public IP.
- HTTPS using Certbot.
- GitHub Actions deploys to EC2 through SSH.
- Environment variables stored on server, not committed to GitHub.

AWS production upgrade

- Move PostgreSQL to AWS RDS.
- Move uploaded files to S3.
- Use ECS or separate EC2 instances for services.
- Use CloudFront for frontend if deploying static or edge assets.
- Use CloudWatch for logs and metrics.
- Use AWS Secrets Manager for secrets later.

GitHub Actions CI/CD flow

51. On pull request: run lint, tests, and type checks.
52. On merge to main: build Docker images.
53. SSH into EC2 server.
54. Pull latest code or Docker images.
55. Run docker compose up -d --build.
56. Run database migrations.
57. Run health checks.

16. Observability with LangSmith and Logs

Production AI systems need visibility. Without traces, you cannot debug why an answer was wrong or why an agent tool failed.

Track in LangSmith

- Prompt input and output.
- Retriever query.
- Retrieved documents.
- Reranker output.
- Tool calls.
- Agent node transitions.
- Latency.
- Token usage.
- Estimated cost.
- Failure reason.

Track in your database

- agent_runs table stores every AI workflow run.
- tool_calls table stores each external action attempt.
- audit_logs table stores user-facing actions and approvals.
- messages table stores conversation history and source references.

Admin dashboard metrics

- Average answer latency.
- Average confidence.
- Number of failed agent runs.
- LLM cost per organization.
- Top unresolved ticket categories.
- Documents with the most citations.
- Questions that returned low confidence.

17. Security and Production Requirements

Authentication and authorization

- Hash passwords using bcrypt or argon2.
- Use short-lived access tokens and refresh tokens.
- Use RBAC middleware for role-protected actions.
- Check organization membership on every request.
- Do not expose internal FastAPI AI service publicly without protection.

Input and file security

- Validate request bodies using Zod on Node API.
- Validate FastAPI payloads using Pydantic.
- Limit file size.
- Allow only safe file types.
- Scan or sanitize document text before indexing.
- Protect against prompt injection in uploaded documents.

AI safety

- Use source-grounded answers.
- Refuse to answer from missing context.
- Require approval for external actions.
- Log every tool call.
- Show citations to the user.

- Keep tenant data isolated during retrieval.
- Do not allow one organization to retrieve another organization's documents.

Operational security

- Use HTTPS.
- Use rate limiting.
- Use CORS carefully.
- Use environment variables for secrets.
- Do not commit API keys.
- Keep Docker images minimal.
- Use backups for PostgreSQL.

18. Testing, Evaluation, and Quality Gates

Backend testing

- Unit tests for auth, roles, tickets, and organizations.
- Integration tests for upload, chat, and ticket triage.
- Tenant isolation tests.
- API validation tests.
- Rate limit tests.

AI evaluation

- Create a small evaluation dataset of company docs and expected answers.
- Test answer correctness.
- Test citation correctness.
- Test hallucination fallback.
- Test retrieval quality.
- Test agent workflow success and failure cases.

```
Example RAG evaluation item
Question: How should we handle payment success but inactive subscription?
Expected source: Billing Runbook
Expected answer must include:
- Check payment provider status
- Verify webhook event
- Check subscription update job
- Re-run sync if payment is paid
Failure if:
- Answer invents steps not in docs
- No source is returned
- Wrong source is cited
```

Frontend testing

- Component tests for important UI components.
- Auth flow tests.
- Chat streaming tests.
- Upload flow tests.
- Approval flow tests.

19. MVP Scope and Advanced Roadmap

Build this first

- Auth and protected dashboard.
- Organization workspace.
- Knowledge source upload.

- RAG ingestion pipeline.
- AI chat with citations.
- Ticket creation.
- AI ticket triage.
- LangSmith tracing.
- Docker Compose deployment.

Add after MVP

- LangGraph incident agent.
- Human approval system.
- GitHub integration.
- Analytics dashboard.
- Workflow builder.
- React Native mobile app.
- Slack/email integration.
- Billing integration later.

Do not build first

- Do not start with billing.
- Do not start with too many integrations.
- Do not start with complex workflow builder.
- Do not start with mobile app before web MVP.
- Do not add unnecessary AI agents before RAG works well.

20. 30-Day Build Plan

Days	Focus	Deliverable
1-2	Project setup	Monorepo, Next.js, Express, FastAPI, Docker Compose.
3-4	Database and auth	PostgreSQL, Prisma, users, organizations, login/register.
5-6	Dashboard foundation	Protected routes, layout, sidebar, organization setup.
7-9	Knowledge upload	Upload UI, file validation, source status, storage.
10-12	RAG ingestion	Text extraction, chunking, embeddings, vector storage.
13-15	RAG chat	Chat UI, streaming, citations, conversation history.
16-18	Ticket module	Ticket CRUD, ticket detail page, status and priority.
19-21	AI triage	Classify ticket, suggest priority, draft reply.
22-24	LangGraph agent	Intent, retrieval, decision nodes, final response.
25-26	Approvals	Pending approval cards, approve/reject flow, audit logs.
27-28	Deployment	Docker, Nginx, EC2, GitHub Actions.
29	Polish	README, screenshots, demo data, seed docs.
30	Portfolio launch	Live link, GitHub repo, LinkedIn post, resume bullets.

First 7 days exact plan

Day 1

- Create GitHub repository.
- Create monorepo structure.
- Add Next.js app.
- Add Express API.
- Add FastAPI service.
- Add Docker Compose.

Day 2

- Setup PostgreSQL and Prisma.
- Create users, organizations, organization_members tables.
- Setup environment variables.

Day 3

- Build register and login APIs.
- Add JWT and refresh token flow.
- Add protected routes on frontend.

Day 4

- Build dashboard layout.
- Add organization setup screen.
- Add sidebar navigation.

Day 5

- Build knowledge source upload UI.
- Build Node upload API.
- Store file locally for MVP.

Day 6

- Build FastAPI ingestion endpoint.
- Extract text from uploaded file.
- Chunk text and save chunks.

Day 7

- Add embeddings and vector search.
- Build basic RAG chat endpoint.
- Display answer and sources in frontend.

21. Demo Scenarios for Recruiters

When applying for jobs, you need strong demo flows. Prepare sample data and show these flows in a video or live demo.

Demo 1: Source-grounded support answer

58. Upload a billing runbook PDF.
59. Ask: Customer paid but subscription is inactive. What should support do?
60. AI answers with steps and citations.
61. Show source cards and confidence score.

Demo 2: Ticket triage

62. Create a new customer ticket.
63. Click AI triage.
64. AI classifies category as Billing.
65. AI sets priority as High.
66. AI drafts customer reply.

Demo 3: Incident workflow

67. Enter: Users are getting 500 errors after latest deployment.
68. Agent searches incident runbook.
69. Agent asks for logs or suggests rollback checklist.

70. Agent drafts incident summary.

Demo 4: Human approval

71. AI proposes creating a GitHub issue.

72. Approval card appears.

73. Admin reviews and approves.

74. System creates issue and logs action.

22. Portfolio, README, LinkedIn, and Resume Positioning

Portfolio description

ResolveAI is a production-grade agentic support and incident resolution platform built with Next.js, Node.js, Express, FastAPI, LangChain, LangGraph, LangSmith, PostgreSQL, vector search, Docker, GitHub Actions, and AWS. It enables SaaS teams to upload internal knowledge, ask source-grounded questions, triage support tickets, create GitHub issue drafts, and run human-approved AI workflows.

GitHub README structure

- Project title and one-line description.
- Live demo link.
- Demo video link.
- Screenshots.
- Problem statement.
- Features.
- Architecture diagram.
- Tech stack.
- AI workflow explanation.
- Database schema overview.
- Local setup commands.
- Docker setup.
- Environment variables.
- Deployment guide.
- Future improvements.

Resume bullets

- Built a production-grade agentic AI SaaS platform using Next.js, Node.js, FastAPI, LangChain, LangGraph, PostgreSQL, vector search, Docker, and AWS.
- Designed and implemented a RAG pipeline with document ingestion, chunking, embeddings, semantic retrieval, source citations, and confidence scoring.
- Developed LangGraph-based support and incident resolution agents with tool calling, human approval checkpoints, and LangSmith observability.
- Implemented multi-tenant SaaS architecture with authentication, RBAC, organization workspaces, ticket management, audit logs, and AI workflow history.
- Deployed the system using Docker, Nginx, GitHub Actions CI/CD, and AWS Linux infrastructure.

LinkedIn project post angle

I built ResolveAI, a production-grade AI-first SaaS platform for support and incident resolution.

This was not just another chatbot.

The system includes:

- Multi-tenant SaaS architecture
- RAG pipeline with source citations
- LangGraph agent workflows
- Human approval before risky actions
- Ticket triage and incident support
- Node.js API gateway and FastAPI AI service
- Docker, CI/CD, and AWS deployment

The goal was to build the kind of AI product real teams could actually use.

23. Starter Commands and Environment Variables

Initial setup commands

```
mkdir resolve-ai
cd resolve-ai
git init

mkdir -p apps services packages infra docs

# Next.js app
cd apps
npx create-next-app@latest web --typescript --tailwind --eslint --app
cd ..

# Node.js API
mkdir -p services/api
cd services/api
npm init -y
npm install express cors helmet morgan dotenv jsonwebtoken bcrypt zod prisma @prisma/client
npm install -D typescript ts-node-dev @types/node @types/express @types/cors @types/jsonwebtoken
@types/bcrypt
npx tsc --init
npx prisma init
cd ../../

# FastAPI AI service
mkdir -p services/ai-service
cd services/ai-service
python3 -m venv .venv
source .venv/bin/activate
pip install fastapi uvicorn langchain langgraph langsmith pydantic python-dotenv
pip install qdrant-client psycpg[binary] pypdf tiktoken
cd ../../
```

Example .env file

```
# General
NODE_ENV=development
APP_URL=http://localhost:3000
API_URL=http://localhost:4000
AI_SERVICE_URL=http://ai-service:8000

# Database
DATABASE_URL=postgresql://resolveai:resolveai@postgres:5432/resolveai

# Auth
JWT_ACCESS_SECRET=change_me_access
JWT_REFRESH_SECRET=change_me_refresh
ACCESS_TOKEN_EXPIRES_IN=15m
REFRESH_TOKEN_EXPIRES_IN=7d

# Redis
REDIS_URL=redis://redis:6379

# AI
OPENAI_API_KEY=your_key_here
LANGCHAIN_API_KEY=your_langsmith_key_here
LANGCHAIN_TRACING_V2=true
LANGCHAIN_PROJECT=resolveai

# Vector DB
```

```
QDRANT_URL=http://qdrant:6333
QDRANT_COLLECTION=resolveai_chunks
```

```
# Storage
STORAGE_DRIVER=local
S3_BUCKET=
AWS_ACCESS_KEY_ID=
AWS_SECRET_ACCESS_KEY=
AWS_REGION=us-east-1
```

```
# GitHub integration
GITHUB_CLIENT_ID=
GITHUB_CLIENT_SECRET=
```

Local run commands

```
# Run all services
docker compose up --build

# Run frontend only
cd apps/web
npm run dev

# Run Node API only
cd services/api
npm run dev

# Run FastAPI only
cd services/ai-service
source .venv/bin/activate
uvicorn app.main:app --reload --port 8000

# Prisma migration
cd services/api
npx prisma migrate dev --name init
npx prisma studio
```

24. Final Checklist

MVP completion checklist

- User can register and login.
- User can create an organization.
- User can upload a document.
- System indexes document chunks.
- User can ask questions from knowledge base.
- AI returns answer with citations.
- User can create a support ticket.
- AI can triage a ticket.
- Agent run is stored in database.
- LangSmith trace exists for AI call.
- App runs locally through Docker Compose.
- App is deployed live on AWS.
- GitHub README is complete.
- Portfolio page has screenshots and live link.

Advanced completion checklist

- LangGraph incident workflow works.
- Human approval flow works.
- GitHub issue draft generation works.
- GitHub issue creation after approval works.

- Analytics dashboard shows AI usage.
- Role-based permissions are enforced.
- Tenant isolation is tested.
- CI/CD deploys automatically.
- React Native companion app is started.

Final advice

Build this project in layers. First make the SaaS foundation. Then make RAG work properly. Then add LangGraph workflows. Then add approvals and integrations. A clean, deployed MVP with strong documentation is better than an unfinished huge system.

Your final positioning should be simple: I build AI-first full-stack products, not just AI demos.